

Experiment 7

Arman Imam

EEE3342C

25Fall 0011

Equipment:

Computer with Xilinx ISE software, Vivado 2017.4 specifically, and Artix-7 FPGA Board

Objective:

The objective of experiment 7 is to introduce the user how to implement ROM and RAM in Verilog through Vivado. For part 1, the user is asked to create a 4 bit register using an SR latch based D flip flop, along with a 4 bit register using posedge always block D flip flops. The user is also asked to screenshot both waveforms for each. For part 2, The user is asked to create a 4 bit PISO Shift Register, and screenshot the waveform and schematic. For part 3, the user is asked to create a 8x8 bit register ROM module, where the user is given 8 prefilled registers. The user will need to create the RTL file for the ROM, screenshot the waveform, and take pictures of the FPGA for each register. For part 4, the user is asked to do make a 8x4 bit register RAM module, where the deliverables are the same as part 3, except the user is prefilling 4 registers instead of 8. For part 5, the user is asked to set up a ROM and RAM heterogeneous architecture, where the user needs to submit the RTL for RAM, ROM, and the top level heterogenous memory modules. The user also needs to take a screenshot of the waveform and take pictures of the FPGA board for each register.

Design Steps:

I began by creating a single project file, where I created the specified source files for each part. For part 1, I was told by the Lab TA's to resend the screenshots I've had from part 5 of experiment 6, due to how similar the question. For the rest of the parts, I've created the source files for the 4-bit PISO Shift Register, 8x8 bit register ROM module, 8x4 bit register RAM module, and the required Heterogeneous Memory Architecture modules(ROM, RAM, Heterogeneous Memory), I've created a test bench for each source file, along with constraints for the FPGA board.

Part 1: WAS TOLD BY TA TO PUT EXPERIMENT 6 PART 5 data here

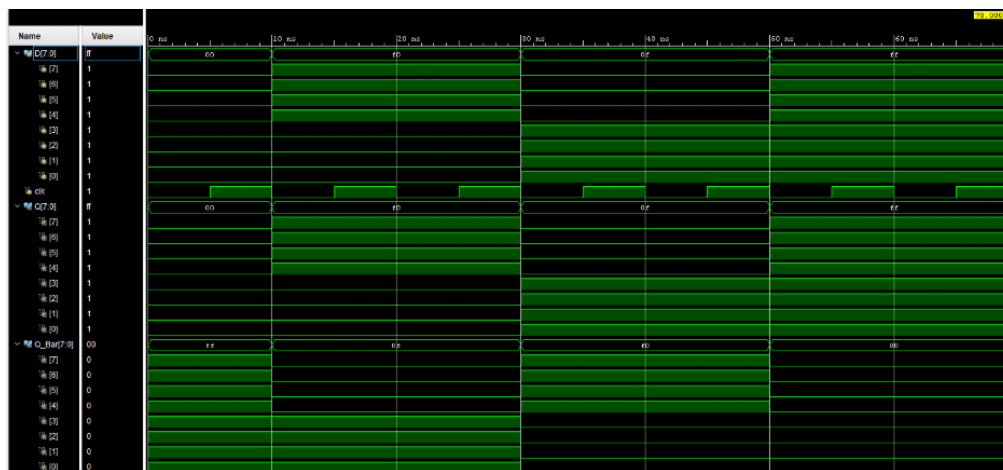
- Verilog RTL of 4-bit register using SR-latch based D flip flops
- Verilog RTL of 4-bit register using posedge always-block D flip flops

Verilog RTL of 8-bit dual edge-triggered flip flop: The source code for the 8-bit dual edge-triggered flip flop has 9 inputs([7:0] D, clk) and 16 outputs([7:0] Q, [7:0] Q_Bar). I've used a simple D flip flop logic for this design.

```
1  `timescale 1ns / 1ps
2  //Tasked for 4-bit register using SR-latch based D flip flops
3  //HOWEVER, TA asked for Repeat from Last part from Lab6
4  //8 bit dual edge D flip flop
5  module part1_l(
6      input [7:0] D,
7      input clk,
8      output reg [7:0] Q, Q_Bar
9  );
10
11     always @(clk) begin
12         Q <= D;
13         Q_Bar = ~Q;
14     end
15
16 endmodule
```

- Screenshot of both waveforms for 4-bit register variations (must match values)

When testing the fpga board, I had to disable the “Q_bar” output due limited number of leds. Since I’ve used a D flip flop design, flipping a switch would turn on an LED.



Part 2:

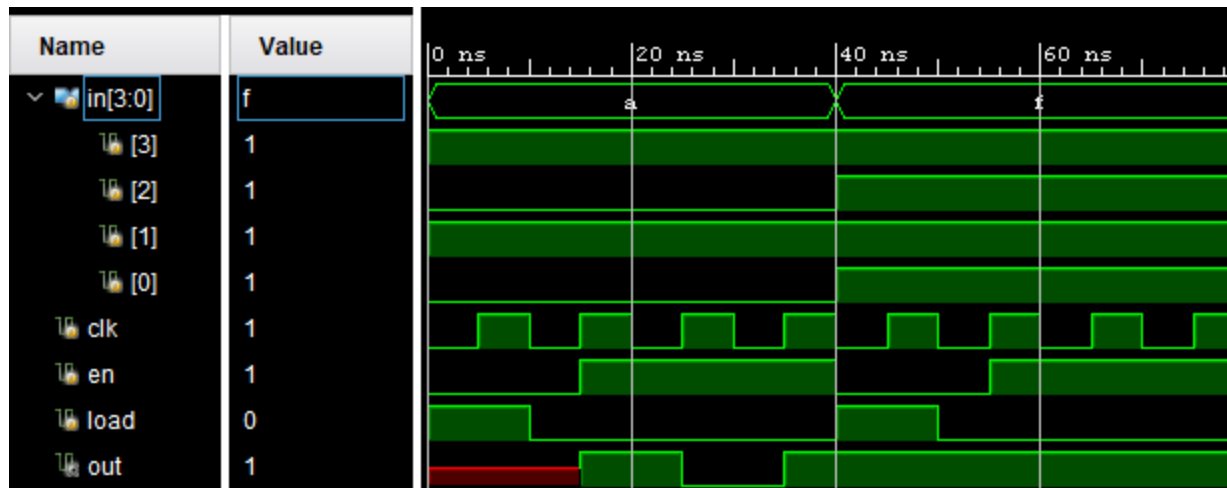
- Verilog RTL of 4-bit PISO shift register

The source code for the 4 bit PISO Shift Register has 7 inputs(4 bit in, clk, en, load), and 1 output(reg out). When load = 1, whatever is in “[3:0] in”, will be sent into the register “q[3:0].” When en = 1, the values of each bit in q, will “shift” downwards, with the first bit being 0, and the output being the last value of q: q[3]. If load and en = 0, then then nothing happens.

```
1 | | `timescale 1ns / 1ps
2 | |
3 | | //Part 2
4 | | //Emulating D flip flop
5 | | //4 bit shift register
6 | | module part2(
7 | |     input [3:0] in,
8 | |     input clk, en, load,
9 | |     output reg out
10 | | );
11 | |     reg q[3:0];
12 | |
13 | |     always @(posedge clk)begin
14 | |         //LOAD stage
15 | |         if (load == 1) begin
16 | |             q[0] <= in[0];
17 | |             q[1] <= in[1];
18 | |             q[2] <= in[2];
19 | |             q[3] <= in[3];
20 | |         end
21 | |         //Shift Stage
22 | |         if (en == 1) begin
23 | |             q[0] <= 0;
24 | |             q[1] <= q[0];
25 | |             q[2] <= q[1];
26 | |             q[3] <= q[2];
27 | |             out <= q[3];
28 | |         end
29 | |     end
30 | | endmodule
```

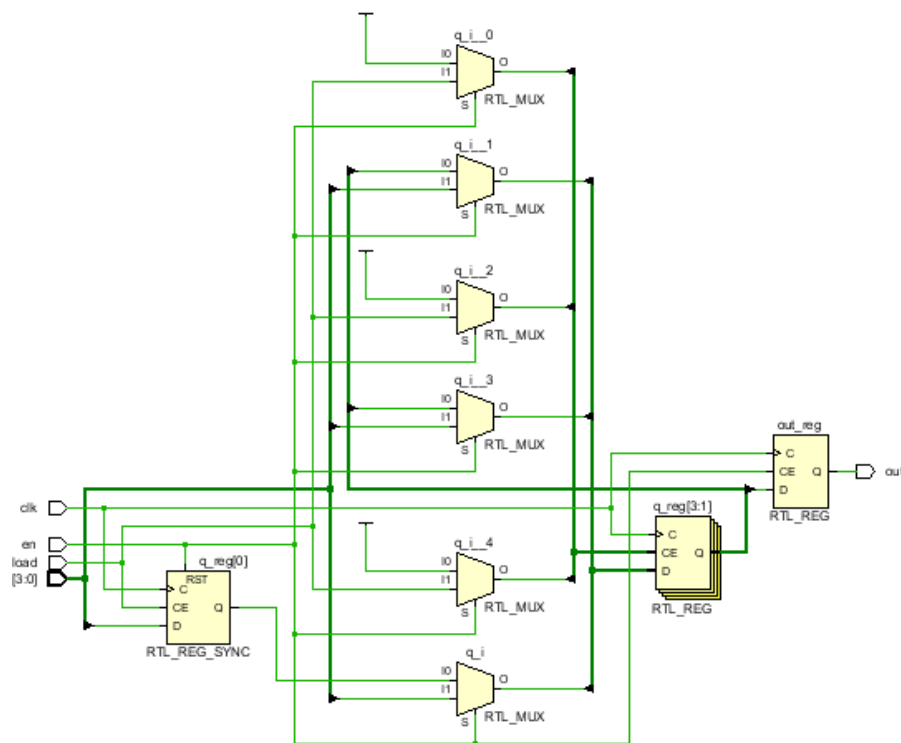
- Waveform screenshot of 4-bit PISO shift register based on example

For 40 ns, I've made [3:0]in = {1010}. For when load = 1 for 10 ns, the inputs are placed into q, though out is undefined for not having a yet. When en becomes 1 at posedge, the shift occurs, and out becomes 1, and becomes 0 at the next posedge. After 40 ns, I've set [3:0]in = {1111}, After setting load = 1, and then en afterwards, the out is set to 1 for 3 clk cycles.



- Screenshot of 4-bit PISO schematic

The schematic provides multiple muxes, and registers, which is to be expected.



Part 3:

- Verilog RTL of ROM

The source code for this 8x8 register ROM module has 3 inputs(3 bit addr), and 16 outputs(16 bit data). The mem register is able to contain 8 sets of 8 bit data. For each mem, I've prefilled the values provided by the experiment brief. The addr retrieves one of the registers for output.

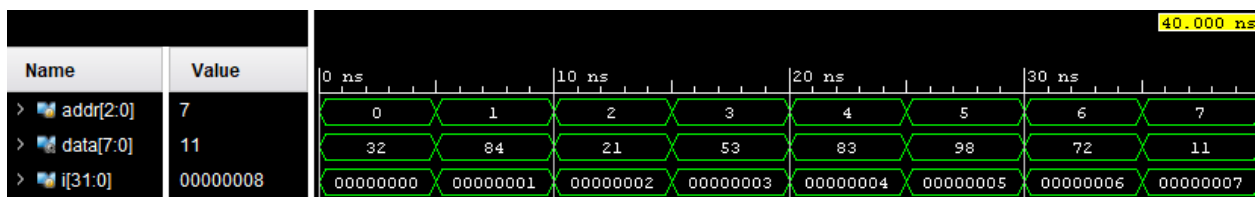
```

1  `timescale 1ns / 1ps
2
3  module part3(
4      input [2:0] addr,
5      output reg [7:0] data
6  );
7      reg [7:0] mem [7:0];
8
9      initial begin
10         mem[0] = 8'h32;
11         mem[1] = 8'h84;
12         mem[2] = 8'h21;
13         mem[3] = 8'h53;
14         mem[4] = 8'h83;
15         mem[5] = 8'h98;
16         mem[6] = 8'h72;
17         mem[7] = 8'h11;
18     end
19     always @(*) begin
20         data = mem[addr];
21     end
22 endmodule

```

- Waveform screenshot of ROM working with stated metrics

For the waveform, I've tested every iteration of the 3 bit addr input. The data matches the prefilled values from the source code, which is expected.

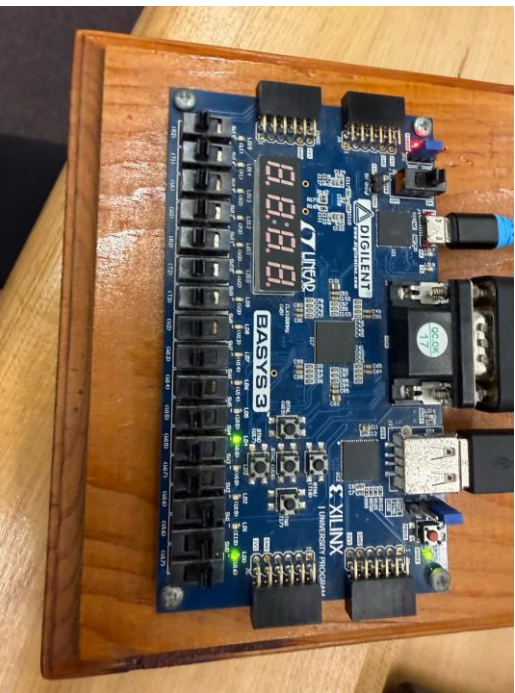


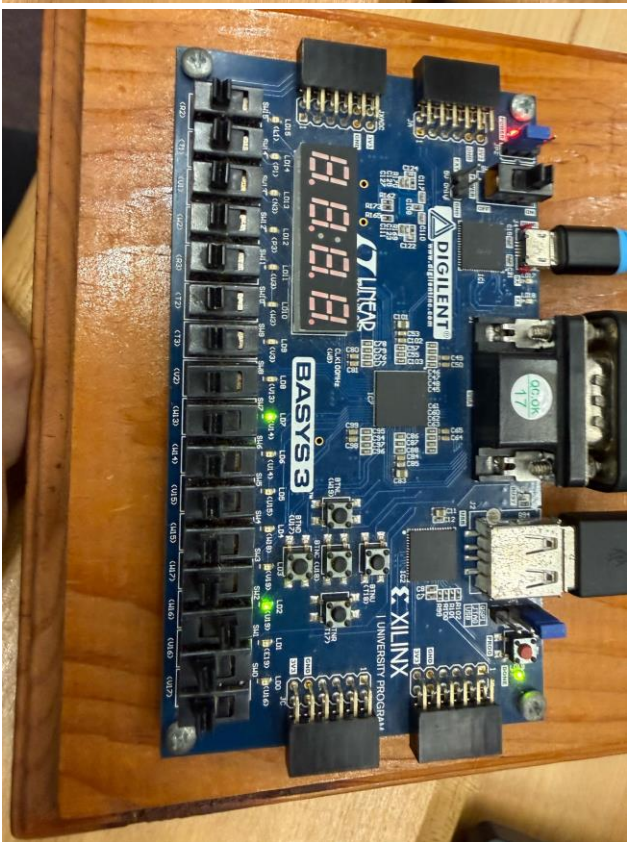
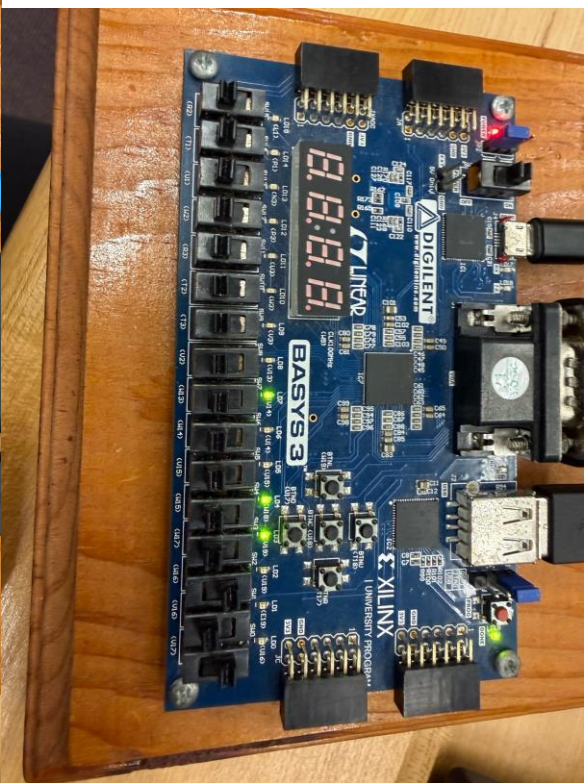
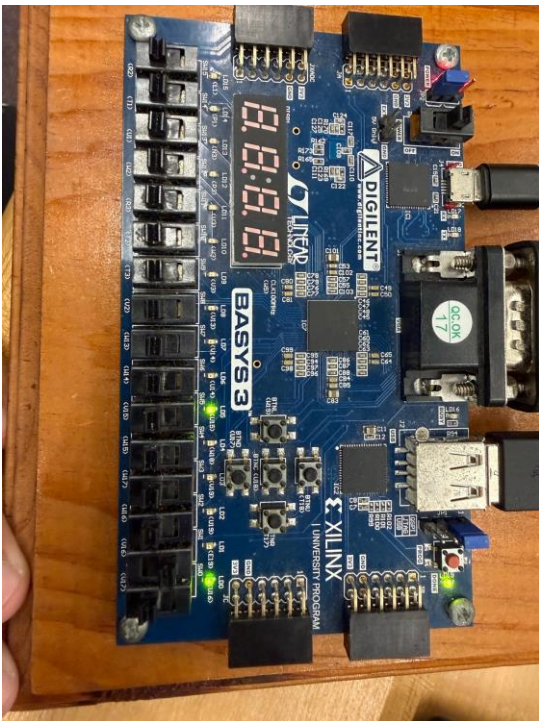
- Pictures of FPGA working for each register within ROM

On this fpga board, I've tested every addr combination, where the LEDs reflect on the specific addr combination. In the source code, the ROM values are already prefilled. For

example, when addr = 000, 8'h32, or 8b00110010 will be outputted and reflected on the fpga board.

Name	Direction	Neg Diff Pair	Package Pin	Fixed	Bank	I/O Std
▼  All ports (11)						
▼  addr (3)	IN			✓	14	LVC MOS33* ▼
 addr[2]	IN		V17 ▼	✓	14	LVC MOS33* ▼
 addr[1]	IN		V16 ▼	✓	14	LVC MOS33* ▼
 addr[0]	IN		W16 ▼	✓	14	LVC MOS33* ▼
▼  data (8)	OUT			✓	14	LVC MOS33* ▼
 data[7]	OUT		V14 ▼	✓	14	LVC MOS33* ▼
 data[6]	OUT		U14 ▼	✓	14	LVC MOS33* ▼
 data[5]	OUT		U15 ▼	✓	14	LVC MOS33* ▼
 data[4]	OUT		W18 ▼	✓	14	LVC MOS33* ▼
 data[3]	OUT		V19 ▼	✓	14	LVC MOS33* ▼
 data[2]	OUT		U19 ▼	✓	14	LVC MOS33* ▼
 data[1]	OUT		E19 ▼	✓	14	LVC MOS33* ▼
 data[0]	OUT		U16 ▼	✓	14	LVC MOS33* ▼





Part 4:

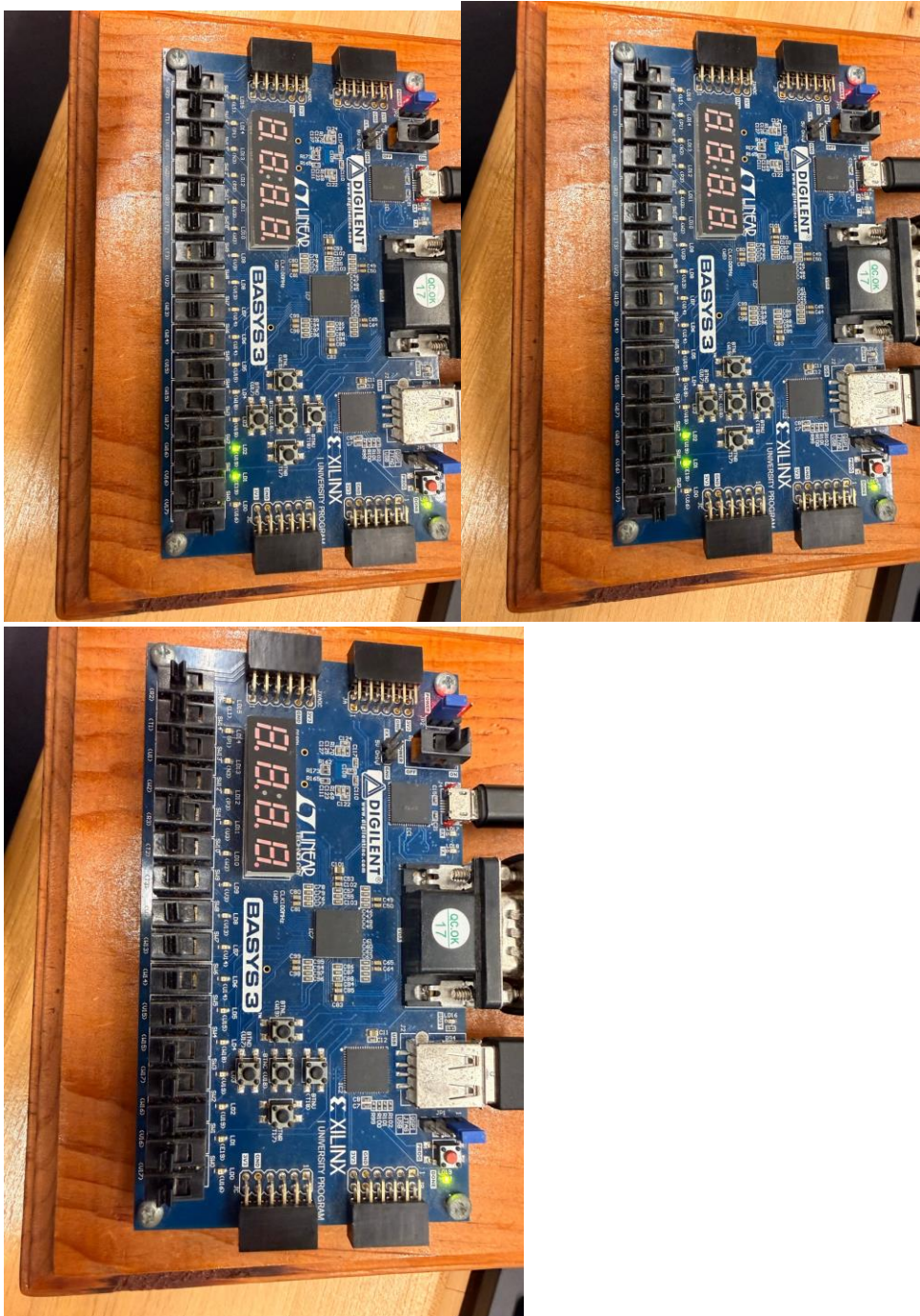
- Verilog RTL of RAM

The source code for this 8x4 bit register RAM module has 10 inputs(4 bit addr, 4 bit in, clk, en) and 4 outputs (4 bit out). The mem register is able to contain 4 sets of 4 bit data. When en = 0, the source code will act as ROM, since there are already prefilled values for mem. I will use the addr to choose with 4 bit mem to choose from and output it. When en = 1, whatever value is in 4 bit in, will replace the register that the addr points to, and then output it.

```
1  |      `timescale 1ns / 1ps
2  |      //8x4bit RAM
3  |      module part4(
4  |          input [2:0] addr,
5  |          input [3:0] in,
6  |          input clk, en,
7  |          output reg [3:0] out
8  |      );
9  |      reg [7:0] mem [3:0];
10 |      always @(posedge clk)begin
11 |          if(en == 1)begin
12 |              mem[addr] <= in;
13 |              out <= in;
14 |          end
15 |          else begin
16 |              out <= mem[addr];
17 |          end
18 |      end
19 |
20 |  endmodule
```

- Waveform screenshot of RAM working with stated metrics

For 50 ns, I've outputted all the registers currently residing in the mem array, while en = 0. When I made en = 1, I've set the address's and input every 10 ns, which I've done for 3 registers. For example, I've put addr = 4'h0, and in=4'ba;. After changing the 3 of the registers, I've set en back to 0, and checked 3 of the addresses, to show that the register was changed.



Part 5:

- Verilog RTL of RAM module

The source code for the 4bit RAM module has 8 inputs(2 bit addr, 4 bit in, clk, en) and 4 outputs(4bit out). The RAM module runs the same as the RAM module from the previous parts, except the RAM memory is preloaded with data(mem[0] = 4'b'0)... in a way that's ordered inverted to the ROM module.

```
34 //4x4bit RAM
35 module RAM_4bit(
36     input [1:0] addr,
37     input [3:0] in,
38     input clk, en,
39     output reg [3:0] out
40 );
41     reg [3:0] mem [3:0];
42     initial begin
43         mem[0] = 4'h0; // RAM[0] = ROM[3]
44         mem[1] = 4'h1; // RAM[1] = ROM[2]
45         mem[2] = 4'h2; // RAM[2] = ROM[1]
46         mem[3] = 4'h3; // RAM[3] = ROM[0]
47     end
48     always @(posedge clk)begin
49         if(en == 1)begin
50             mem[addr] <= in;
51             out <= in;
52         end
53         else begin
54             out <= mem[addr];
55         end
56     end
57 endmodule
```

- Verilog RTL of ROM module

The source code for the 4bit ROM module has 2 inputs(2bit addr) and 4 outputs(4bit data). This ROM module runs the same as the ROM module from the previous parts, except the the value from mem[0] to mem[3] is 4'h3, 4'h2, 4'h1, 4'h0. When outputting to the RAM module, the data will move invertedly(mem[0] from ROM will go to mem[3] in RAM).

```
17 //4x4 bit ROM
18 module ROM_4bit(
19     input [1:0] addr,
20     output reg [3:0] data
21 );
22     reg [3:0] mem [3:0];
23     initial begin
24         mem[0] = 4'h3;
25         mem[1] = 4'h2;
26         mem[2] = 4'h1;
27         mem[3] = 4'h0;
28     end
29     always @(*) begin
30         data = mem[addr];
31     end
32 endmodule
```

- Verilog RTL of Top-level heterogenous memory

The source code for the top level heterogeneous memory has 8 inputs(en, clk, 2bit addr, 4bit in) and 4 outputs(4bit out). I've created two wires in the module, where the 2bit newAddr takes the addr value from the ROM, and will be inputted in the RAM module. And the 4bit data takes the data from the ROM at the specified address.

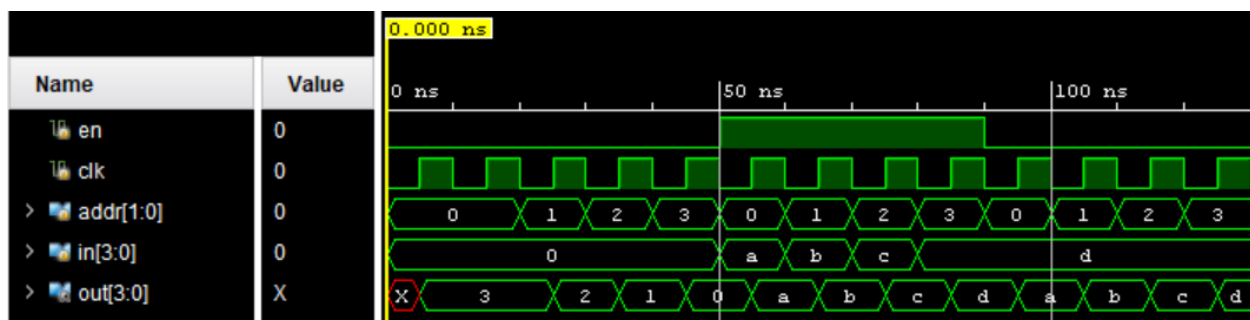
```

1  `timescale 1ns / 1ps
2  //Heterogeneous Memory Module
3  module part5(
4      input en, clk,
5      input [1:0] addr,
6      input [3:0] in,
7      output [3:0] out
8  );
9      wire [1:0] newAddr;
10     wire [3:0] data;
11     ROM_4bit ABT(.addr(addr), .data(data));
12     assign newAddr = data[1:0];
13     RAM_4bit ACT(.addr(newAddr), .in(in), .clk(clk), .en(en), .out(out));

```

- Waveform screenshot of memory architecture working with stated metrics

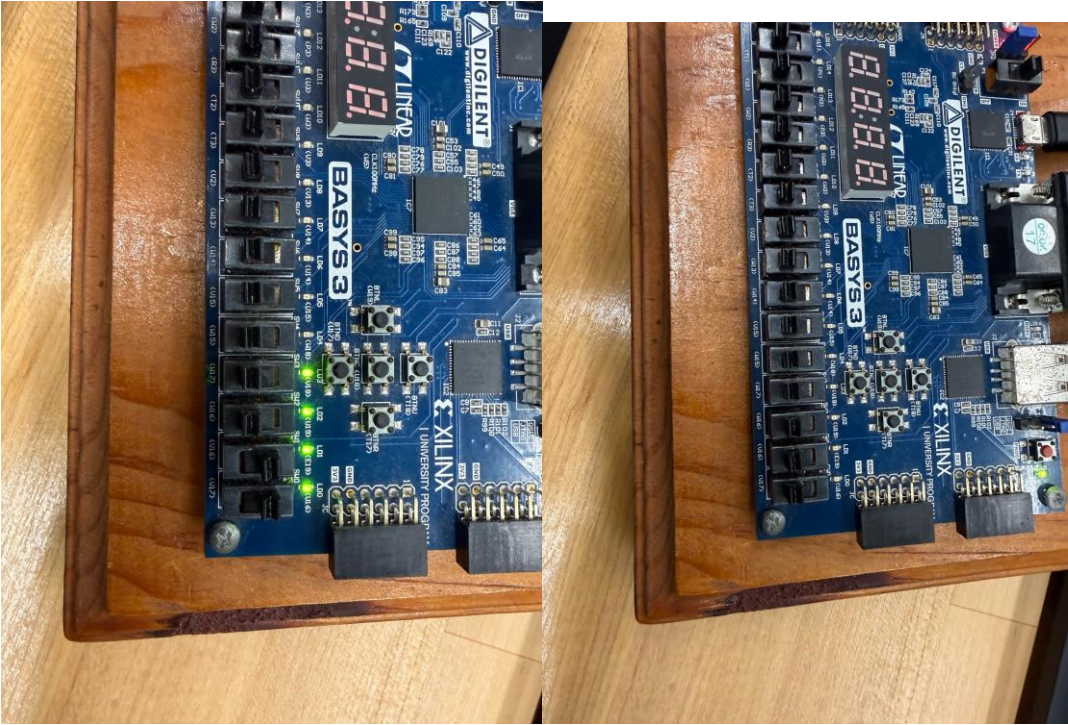
Before En is enabled, the test bench reads through what's already in the architecture. Once En is enabled, I've set addr 2'b00 to 4'hA, and 2'b01 to 4'hB, 2'b10 to 4'hC, and 2'b11 to 4'hD. After En is disabled, the test bench reads through the memory oncemore to reflect the changes.



- Pictures of FPGA working for each register within memory architecture

In this test, I've set the enable to 1, and then address to 011 with 3 switches, and inputted 1111 for the 4 input switches. After turning the enable to 0, the LEDs for 1111 light up for when the address is set to 011, which is expected for the RAM module.

Name	Direction	Neg Diff Pair	Package Pin	Fixed	Bank	I/O Std
All ports (12)						
addr (2)	IN			✓	14	LVC MOS33*
addr[1]	IN		V16	✓	14	LVC MOS33*
addr[0]	IN		V17	✓	14	LVC MOS33*
in (4)	IN			✓	14	LVC MOS33*
in[3]	IN		W14	✓	14	LVC MOS33*
in[2]	IN		V15	✓	14	LVC MOS33*
in[1]	IN		W15	✓	14	LVC MOS33*
in[0]	IN		W17	✓	14	LVC MOS33*
out (4)	OUT			✓	14	LVC MOS33*
out[3]	OUT		V19	✓	14	LVC MOS33*
out[2]	OUT		U19	✓	14	LVC MOS33*
out[1]	OUT		E19	✓	14	LVC MOS33*
out[0]	OUT		U16	✓	14	LVC MOS33*
Scalar ports (2)						
clk	IN		W5	✓	34	LVC MOS33*
en	IN		T3	✓	34	LVC MOS33*



Q&A:

- What is the difference between a register and a shift register?

A register stores data temporarily, while a shift register stores data, and moves it bit by bit.

- What two types of data conversions does a shift register perform?

A serial to parallel conversion, where the data is inputted one bit at a time, and outputted all at once, and a parallel to serial conversion, where the opposite happens: data is inputted all at once and outputted one at a time.

- How many clock cycles would it take to serially output data from a 32-bit shift register?

It would take 32 clock cycles, since every 1 bit is moved once for each clock cycle.

- What is the difference between ROM and RAM?

ROM is Read Only Memory; used to store and read data, and is often used in firmware.

RAM is Random Access Memory; used for reading and writing data frequently, and is often used for holding storage of data temporarily.

- Why would you want to use flip-flop registers over vector-based registers?

Flip flop registers can store and update one bit at a time, whereas vector based registers would be better for handling large amounts of data at once.

- How many registers should I expect to have if the address size of RAM is 8-bits?

8 registers would be needed for each bit in the address size of RAM

Test Plan:

Part 1: Mentioned logic for waveform section

```
1  `timescale 1ns / 1ps
2
3  module dual_edge_flip_flop_8bit_sim();
4      reg[7:0] D;
5      reg clk;
6      wire [7:0] Q, Q_Bar;
7
8      dual_edge_flip_flop_8bit PARTFIVE(.D(D), .clk(clk), .Q(Q), .Q_Bar(Q_Bar));
9      initial begin
10         clk = 0;
11         D = 8'b00000000;
12         #10;
13         D = 8'b11110000;
14         #20;
15         D = 8'b00001111;
16         #20;
17         D = 8'b11111111;
18         #20;
19         $finish;
20     end
21     always #5 clk = ~clk;
22
23
24 endmodule
```

Part 2: Mentioned logic for waveform section

```

1  `timescale 1ns / 1ps
2  module part2_sim();
3      reg [3:0] in;
4      reg clk, en, load;
5      wire out;
6
7      part2 TESTONE(.in(in), .clk(clk), .en(en), .load(load), .out(out));
8      initial begin
9          //in is 1010
10         clk = 0;
11         in = 1010;
12         en = 0;
13         load = 1;
14
15         #10;
16         load = 0;
17         en = 0;
18         #5;
19         en = 1;
20         #25;
21         //in now 1111
22         in = 1111;
23         en = 0;
24         load = 1;
25
26         #10;
27         load = 0;
28         en = 0;
29         #5;
30         en = 1;
31         #25;
32         $finish;
33     end
34     always #5 clk = ~clk;
35 endmodule

```

Part 3: Mentioned logic for waveform section

```

1  `timescale 1ns / 1ps
2
3  module part3_sim();
4      reg [2:0] addr;
5      wire [15:0] data;
6      integer i;
7
8      part3 TEST2(.addr(addr), .data(data));
9      initial begin
10         for(i=0; i<2**3; i=i+1)begin
11             {addr} = i;
12             #5;
13         end
14         $finish;
15     end
16 endmodule

```

Part 4: Mentioned logic for waveform section

```

1  `timescale 1ns / 1ps
2
3  module part4_sim();
4      reg [3:0] addr, in;
5      reg clk, en;
6      wire [3:0] out;
7      integer i;
8      part4 TEST(.addr(addr), .in(in), .clk(clk), .en(en), .out(out));
9      initial begin
10         en = 0;
11         clk = 0;
12         //Testing all addr for when en = 0;
13         for(i = 0; i<2**2; i=i+1)begin
14             addr = i;
15             #10;
16         end
17
18         //Writing new values for when en=1;
19         en = 1;
20         #10;
21         addr = 4'h0; in = 4'ha;
22         #10;
23         addr = 4'h1; in = 4'hb;
24         #10;
25         addr = 4'h2; in = 4'hc;
26
27         //Reading what the values are currently.
28         #10;
29         en = 0;
30         addr = 4'h0;
31         #10;
32         addr = 4'h1;
33         #10;
34         addr = 4'h2;
35         #10;
36         $finish;
37     end
38     always #5 clk = ~clk;
39
40 endmodule

```

Part 5: Mentioned logic for waveform section


```

1  `timescale 1ns / 1ps
2  module part5_sim();
3      reg en, clk;
4      reg [1:0] addr;
5      reg [3:0] in;
6      wire [3:0] out;
7
8      part5 DUT(.en(en), .clk(clk), .addr(addr), .in(in), .out(out));
9      initial clk = 0;
10     always #5 clk = ~clk;
11     initial begin
12         en = 0;
13         addr = 2'b00;
14         in = 4'h0;
15         #10;
16         // Read through ROM -> RAM mapping
17         addr = 2'b00; en = 0;
18         #10;
19         addr = 2'b01; en = 0;
20         #10;
21         addr = 2'b10; en = 0;
22         #10;
23         addr = 2'b11; en = 0;
24         #10;
25         // Write operations
26         addr = 2'b00; in = 4'hA; en = 1;
27         #10;
28         addr = 2'b01; in = 4'hB; en = 1;
29         #10;
30         addr = 2'b10; in = 4'hC; en = 1;
31         #10;
32         addr = 2'b11; in = 4'hD; en = 1;
33         #10;
34         // Read back written values
35         addr = 2'b00; en = 0;
36         #10;
37         addr = 2'b01; en = 0;
38         #10;
39         addr = 2'b10; en = 0;
40         #10;
41         addr = 2'b11; en = 0;
42         #10;
43         $finish;
44     end
45 endmodule

```

Conclusion

This experiment was a success. The results were as expected for all parts of the experiment. I've had issues conceptually on how the PISO Shift register and wasn't able to

figure it out on my own; needed assistance from lab TA's. After receiving help, I was able to understand it. Implementing ROM and RAM was through Vivado and FPGA has furthered my understanding of how its made, and the specifics of it.